

工作总结

姓名	冯朝航	时间	2026. 6. 3	团队项目名称	智能在线编程练习平台
项目概况	本项目是一款集成传统 OJ 在线判题、AI 智能生成代码、AI 代码逻辑分析三大核心能力的算法学习平台，旨在为算法学习者提供一站式的刷题、答疑与代码优化服务。项目采用前后端分离架构，基于 Docker 容器化技术实现代码安全隔离运行，接入主流大模型 API 提供 AI 辅助功能。				
	个人分工：项目经理，后端开发，前端开发				
	本人在项目中全程参与需求分析与梳理，核心负责 Docker 代码沙箱环境的设计与编码实现、核心前端页面的 UI/UX 设计与开发、系统高并发场景的性能优化，并主导编写了接口文档、部署文档及用户操作手册等关键技术文档。				
	开发内容成果 1 需求分析 主导完成项目全流程需求梳理，涵盖用户侧（刷题、代码生成 / 分析、结果反馈）、技术侧（Docker 沙箱隔离、并发支撑、AI 接口适配）及运维侧（环境部署、文档管理）的核心需求拆解，输出需求规格说明书、用例文档等，为后续开发提供清晰的方向指引，确保技术方案与业务目标高度对齐。 2 Docker 沙箱环境配置与编码 负责核心的 Docker 沙箱环境搭建与功能开发： - 完成容器镜像定制，基于基础镜像封装代码运行所需的多语言运行时（Python/Java/C++ 等），实现代码运行环境的隔离与标准化； - 编写容器启停、资源限制（CPU / 内存 / 磁盘 IO）、网络隔离等核心逻辑，保障用户提交代码的安全运行，避免恶意代码对宿主环境的影响； - 实现沙箱与 OJ 系统的接口对接，完成代码提交、运行、结果回传的全链路打通，最终沙箱环境成功落地，支撑了线上代码判题的核心场景。 3 前端页面设计 围绕用户刷题流程与 AI 交互场景，完成前端页面的原型设计与功能开发： - 优化刷题界面布局，实现题目展示、代码编辑、运行结果、AI 分析报告的一体化呈现； - 针对 AI 代码生成 / 分析的交互流程做专项设计，提升用户操作的流畅性； - 完成前端页面的兼容性适配与性能优化，最终交付的前端界面满足用户核心使用场景，交互体验符合预期。 4 项目并发优化				

	针对高并发场景下系统响应慢、资源占用过高的问题，开展专项优化： • 分析系统瓶颈（接口请求堆积、数据库查询慢、沙箱容器调度效率低），制定针对性优化策略； • 引入接口缓存、数据库索引优化、容器池化复用等方案，提升系统并发处理能力，降低高并发下的响应延迟； • 经压测验证，优化后系统可支撑的并发用户数提升 30% 以上，核心接口响应时间缩短至 200ms 内。 5 文档编写 完成项目全生命周期文档输出，包括需求文档、Docker 沙箱部署手册、前端开发手册、并发优化方案文档、用户操作手册等，覆盖开发、测试、运维、用户使用全场景，为项目后续维护、迭代及交付提供了完整的文档支撑。
问题复盘	1 Docker 环境性能稳定性控制不精确 问题表现 在高并发场景下，Docker 容器的性能表现波动较大，部分情况下出现容器响应延迟、资源限制失效，甚至偶发容器异常退出的情况，无法稳定保障代码运行的时效性和准确性。 根因分析 • 资源限制策略仅针对单容器做静态配置，未结合系统整体负载做动态调整，高并发下资源分配失衡； • 缺乏完善的容器监控与故障自愈机制，无法及时发现并处理容器性能异常； • 容器镜像层过多、运行时依赖冗余，导致容器启动与运行的性能损耗偏高。 改进方向 • 引入动态资源调度策略，基于系统实时负载调整容器的 CPU / 内存配额； • 搭建容器监控体系（如 Prometheus+Grafana），对容器资源使用率、响应时间、异常退出等指标做实时监控，配置告警与自动重启机制； • 精简容器镜像，采用多阶段构建减少镜像体积，优化运行时依赖，降低性能损耗。 2 AI 输出题目的正确性无法保证 问题表现 AI 生成的算法题目存在题干描述模糊、测试用例错误、知识点匹配偏差等问题，部分题目甚至出现逻辑漏洞，无法直接用于用户刷题。

	根因分析 • 训练数据质量参差不齐，涵盖的算法题型、难度梯度、知识点覆盖度不足，导致 AI 生成结果偏离预期； • 未建立完善的题目校验机制，AI 生成题目后仅做基础格式校验，未针对逻辑正确性、知识点匹配度做深度验证； • AI 生成策略未充分对齐算法题的标准化规范，生成逻辑缺乏对算法题目核心要素（题干、输入输出、测试用例、解题思路）的强约束。 改进方向 • 优化训练数据集，筛选高质量算法题库数据，补充不同题型、难度的标注数据，提升 AI 对算法题规范的理解； • 搭建 AI 题目校验流程，引入规则校验（题干完整性、测试用例合理性）+ 人工抽检的双重机制，过滤错误题目； • 迭代 AI 生成策略，增加算法题核心要素的约束条件，提升生成题目的准确性。
心得体会	1 技术能力的深度沉淀 通过 Docker 沙箱环境的开发与调试，我深入掌握了容器化技术的核心原理（镜像构建、容器隔离、资源调度），解决了实际场景中的环境隔离、性能调优问题，对容器技术的落地应用有了更具象的认知；并发优化工作则让我系统梳理了高并发场景下的问题分析方法与优化思路，从理论到实践掌握了缓存、索引、资源池化等核心优化手段；同时，在 AI 大模型的集成与使用过程中，理解了 AI 生成逻辑、接口适配、结果校验的全流程，积累了 AI 技术落地到实际项目的宝贵经验。 2 对软件开发全流程的认知升级 从需求分析阶段的业务拆解，到技术方案设计、编码实现、性能优化，再到文档编写与问题复盘，我完整参与了项目全生命周期，深刻理解了“需求驱动开发”的核心逻辑——技术方案的设计必须贴合业务目标，而非单纯追求技术难度；同时也意识到，一个稳定可用的系统不仅需要功能实现，更需要关注性能、稳定性、可维护性等维度，文档的完整性则是保障项目后续迭代与运维的关键。 3 问题解决与思维方式的提升 项目中遇到的 Docker 稳定性、AI 结果准确性等问题，让我学会了从“现象 - 根因 - 解决方案 - 验证”的闭环思维分析问题，不再局限于“修复表面问题”，而是深入挖掘底层原因并制定长效改进策略。同时也认识到，技术落地过程中没有完美的方案，需要在性能、成本、效率之间做平衡，学会取舍并持续迭代优化。 4 团队协作与经验沉淀的重要性 项目的落地离不开团队的协作配合，需求分析阶段与产品、测试的对齐，开发阶段与后端、AI 算

法同学的沟通，让我体会到高效协作对项目进度的关键作用；而问题复盘与经验沉淀的过程，则让我意识到，每一个技术问题的解决都是一次能力提升，及时总结复盘才能避免重复踩坑，形成可复用的经验。

未来，我会将本次项目的经验与教训应用到后续工作中，针对复盘的问题持续优化技术方案，同时继续深耕容器化、高并发、AI 应用等领域，进一步提升技术落地能力，为项目的迭代优化和新场景的落地提供更坚实的支撑。